SE-I - NOTES

UNIT I:

Introduction: Evolution, Software development projects, Exploratory style of software developments, Emergence of software engineering, Notable changes in software development practices, Computer system engineering.

Software Life Cycle Models: Basic concepts, Waterfall model and its extensions, Rapid application development, Agile development model, Spiral model.

Software Engineering: software engineering the process of designing, developing, testing, and maintaining software. It is a systematic and disciplined approach to software development that aims to create high-quality, reliable, and maintainable software

Characterstics of software:

- ✓ **Efficiency**: It provides a measure of the resource requirement of a software product efficiently.
- ✓ **Reliability**: It assures that the product will deliver the same results when used in similar working environment.
- ✓ **Reusability**: This attribute makes sure that the module can be used in multiple applications.
- ✓ **Maintainability**: It is the ability of the software to be modified, repaired, or enhanced easily with changing requirements.
- ✓ **Portability:**

Software Evolution:

Software Evolution is a term that refers to the process of developing software initially, and then timely updating it for various reasons, i.e., to add new features or to remove obsolete functionalities

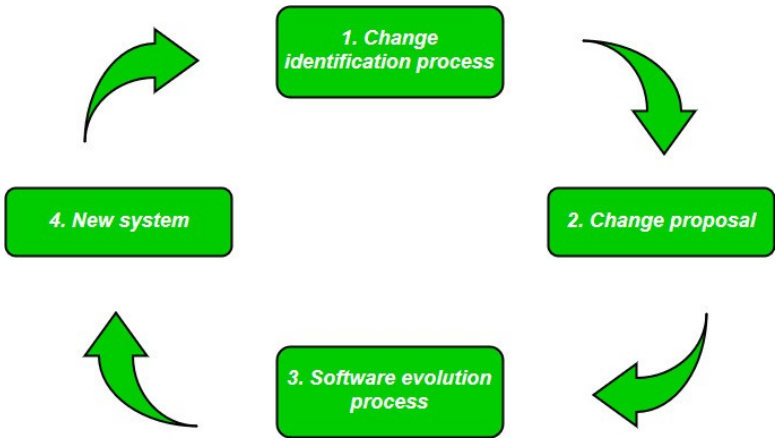
The software evolution process includes fundamental activities of change analysis, release planning, system implementation, and releasing a system to customers.

1. The cost and impact of these changes are accessed to see how much the system is affected by the change and how much it might cost to implement the change.
2. If the proposed changes are accepted, a new release of the software system is planned.
3. During release planning, all the proposed changes (fault repair, adaptation, and new functionality) are considered.
4. A design is then made on which changes to implement in the next version of the system.
5. The process of change implementation is an iteration of the development process where the revisions to the system are designed, implemented, and tested.

Necessity of Software Evolution

Software evaluation is necessary just because of the following reasons:

1. **Change in requirement with time:** With time, the organization's needs and modus Operandi of working could substantially be changed so in this frequently changing time the tools(software) that they are using need to change to maximize the performance.
2. **Environment change:** As the working environment changes the things(tools) that enable us to work in that environment also changes proportionally same happens in the software world as the working environment changes then, the organizations require reintroduction of old software with updated features and functionality to adapt the new environment.
3. **Errors and bugs:** As the age of the deployed software within an organization increases their preciseness or impeccability decrease and the efficiency to bear the increasing complexity workload also continually degrades. So, in that case, it becomes necessary to avoid use of obsolete and aged software. All such obsolete Pieces of software need to undergo the evolution process in order to become robust as per the workload complexity of the current environment.
4. **Security risks:** Using outdated software within an organization may lead you to at the verge of various software-based cyberattacks and could expose your confidential data illegally associated with the software that is in use. So, it becomes necessary to avoid such security breaches through regular assessment of the security patches/modules are used within the software. If the software isn't robust enough to bear the current occurring Cyber attacks so it must be changed (updated).
5. **For having new functionality and features:** In order to increase the performance and fast data processing and other functionalities, an organization need to continuously evolute the software throughout its life cycle so that stakeholders & clients of the product could work efficiently.



Software development projects

Here are nine steps you can follow to develop a software project:

1. Evaluate your project. Assess the feasibility of your project. This includes analyzing its concept, goals, scope and specifications.
2. Specify the requirements. Figure out and document the technical needs of the project. During this phase, it's important to answer questions about who the target user is and what this project solves or makes easier for them.
3. Make a plan. Determine the specific components of the project and make a timeline for the completion of each task. This includes figuring out which tasks depend on others for completion, what resources you need and what your budgetary requirements are.
4. Conceptualize the design. Design the project's architecture and functions. Software architects and engineers typically handle this stage of the software development life cycle.
5. Establish metrics. Set up software metrics to help you continuously track and evaluate your project's progress. Consider using project analytics software to help you regularly collect and analyze your data.
6. Develop the software. Develop and code the project. This step is usually one of the longer phases of the software development life cycle.
7. Conduct testing. Rigorously test and conduct quality control assessments of the software. During the testing phase, it's important to evaluate various aspects of the software, including the quality of the code, compliance with regulations and how well the software meets the established requirements or metrics.
8. Deploy the software. Deploy the software to production. Depending on factors like how complex the software is, you might deploy the program several times, such as deploying it first for pre-release testing.
9. Perform post-production tasks. Conduct updates or maintenance work as needed on the software. This may include updating the software to work on new operating systems, increasing its scale or adding new features.

Changing Nature of Software

The software is an instruction or computer program that when executed provides desired features, function, and performance. A data structure that enables the program to adequately manipulate information and documents that describe the operation and use of the program.

Nowadays, seven broad categories of computer software present continuing challenges for software engineers.

1. **System Software**: System software is a collection of programs that are written to service other programs. Some system software processes complex but determinate, information structures. Other system application processes largely indeterminate data. Sometimes when, the system software area is characterized by the heavy interaction with computer hardware that requires scheduling, resource sharing, and sophisticated process management.
2. **Application Software**: Application software is defined as programs that solve a specific business need. Application in this area processes business or technical data in a way that facilitates business operation or management technical decision-making. In addition to conventional data processing applications, application software is used to control business functions in real-time.
3. **Engineering and Scientific Software**: This software is used to facilitate the engineering function and task. however modern applications within the engineering and scientific area are moving away from conventional numerical algorithms. Computer-aided design, system simulation, and other interactive applications have begun to take a real-time and even system software characteristic.
4. **Embedded Software**: Embedded software resides within the system or product and is used to implement and control features and functions for the end-user and for the system itself. Embedded software can perform limited and esoteric functions or provide significant function and control capability.
5. **Product-line Software**: Designed to provide a specific capability for use by many customers, product-line software can focus on the limited and esoteric marketplace or address the mass consumer market.

6. **Web Application**: It is a client-server computer program that the client runs on the web browser. In their simplest form, Web apps can be little more than a set of linked hypertext files that present information using text and limited graphics. However, as e-commerce and B2B applications grow in importance. Web apps are evolving into a sophisticated computing environment that not only provides a standalone feature, computing function, and content to the end user.
7. **Artificial Intelligence Software**: Artificial intelligence software makes use of a nonnumerical algorithm to solve a complex problem that is not amenable to computation or straightforward analysis. Applications within this area include robotics, expert systems, pattern recognition, artificial neural networks, theorem proving, and game playing.

Computer System Engineering

Computer system engineering encompasses the design, development, and management of computer hardware systems.It is a multidisciplinary field that combines elements of electrical engineering, computer science, and computer engineering.Computer system engineering requires a deep understanding of both hardware and software to create robust and efficient computer systems.

It also involves the integration of hardware components, such as processors, memory, and storage devices, to create efficient and functional computer systems.The role of a computer system engineer is crucial in ensuring that computer systems are not only reliable and secure but also capable of meeting the ever-increasing demands of today's technology-driven world.

.

System architecture

One of the key aspects of computer system engineering is system architecture.System architects are responsible for designing the overall structure of a computer system, including the arrangement of hardware components and the communication between them.They consider factors such as performance, scalability, and reliability when designing the system architecture.

Computer system engineers collaborate with software engineers to design systems that optimise hardware and software interactions, resulting in high-performance computing solutions.

Component selection

Component selection is another critical aspect of computer system engineering.Engineers carefully choose the hardware components that will be used in the system, taking into account factors such as cost, performance, power consumption, and compatibility.They evaluate different options and make informed decisions to ensure that the chosen components meet the system's requirements.

System integration

System integration is the process of combining different hardware components and software modules to create a fully functional computer system.It involves connecting the various components, configuring them, and testing their interactions.System engineers work closely with software engineers during this phase to ensure that the hardware and software components work seamlessly together.

Performance optimisation

Performance optimisation is an ongoing process in computer system engineering. Computer system engineers continuously monitor and analyse the performance of the system, identifying bottlenecks and areas for improvement.They may tweak the system configuration, upgrade hardware components, or optimise software algorithms to enhance the system's performance.

Computer system engineers play a vital role in ensuring the reliability and security of computer systems. They implement measures to protect against hardware failures, software vulnerabilities, and cyber-attacks.

They also design backup and recovery systems to ensure that data is not lost in case of system failures. Computer system engineers are also at the forefront of developing innovative solutions to meet these demands. They constantly explore new technologies,

such as cloud computing and parallel processing, to design systems that can handle complex computational tasks effciently.

1.4 EMERGENCE OF SOFTWARE ENGINEERING

We have already pointed out that software engineering techniques have evolved over many years in the past. This evolution is the result of a series of innovations and accumulation of experience about writing good quality programs. Since these innovations and programming experiences are too numerous, let us briefly examine only a few of these innovations and programming experiences which have contributed to the development of the software engineering discipline.

1.4.1 Early Computer Programming

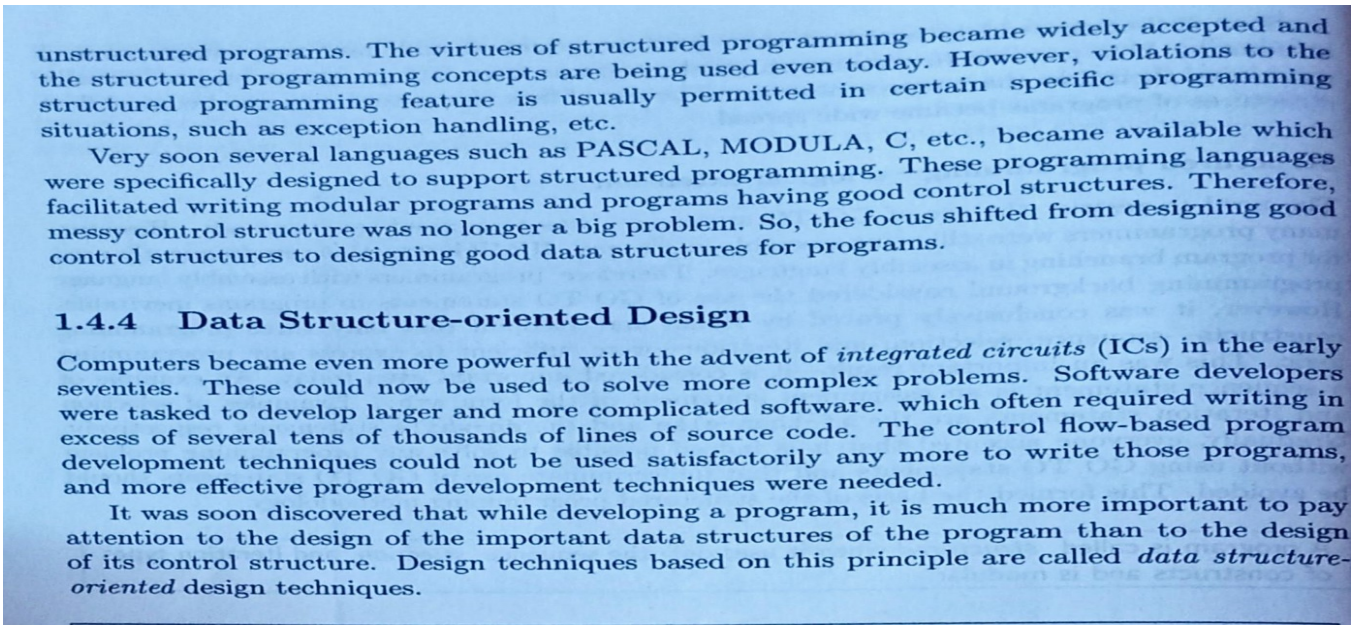
Early commercial computers were very slow and too elementary as compared to today's standards. Even simple processing tasks took considerable computation time on those computers. No wonder that programs at that time were very small in size and lacked sophistication. Those programs were usually written in assembly languages. Program lengths were typically limited to about a few hundreds of lines of monolithic assembly code. Every programmer developed his own individualistic style of writing programs according to his intuition and used this style *ad hoc* while writing different programs. In simple words, programmers wrote programs without formulating any proper solution strategy, plan, or design a jump to the terminal and start coding immediately on hearing out the problem. They then went on fixing any problems that they observed until they had a program that worked reasonably well. We have already designated this style of programming as the *build and fix* (or the *exploratory programming*) style.

1.4.2 High-level Language Programming

Computers became faster with the introduction of the semiconductor technology in the early 1960s. Faster semiconductor transistors replaced the prevalent vacuum tube-based circuits in a computer. With the availability of more powerful computers, it became possible to solve larger and more complex problems. At this time, high-level languages such as FORTRAN, ALGOL, and COBOL were introduced. This considerably reduced the effort required to develop software and helped programmers to write larger programs (why?). Writing each high-level programming construct in effect enables the programmer to write several machine instructions. Also, the machine details (registers, flags, etc.) are abstracted from the programmer. However, programmers were still using the exploratory style of software development. Typical programs were limited to sizes of around a few thousands of lines of source code.

1.4.3 Control Flow-based Design

As the size and complexity of programs kept on increasing, the exploratory programming style proved to be insufficient. Programmers found it increasingly difficult not only to write cost-effective and correct programs, but also to understand and maintain programs written by others. To cope up with this problem, experienced programmers advised other programmers to pay particular attention to the design of a program's *control flow structure*.



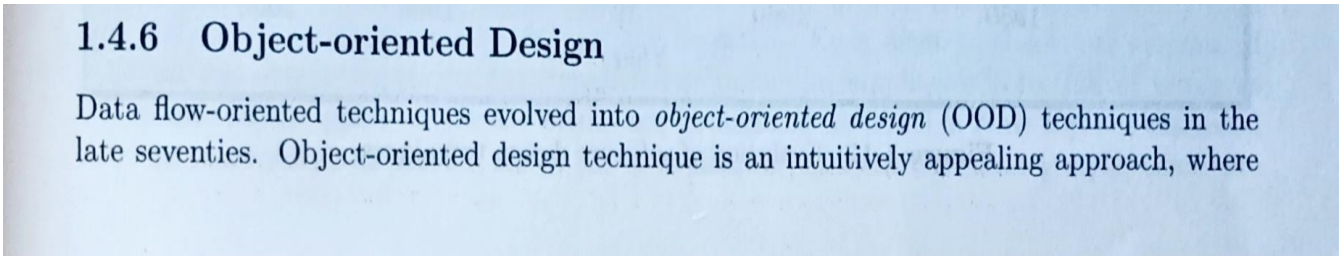
unstructured programs. The virtues of structured programming became widely accepted and the structured programming concepts are being used even today. However, violations to the structured programming feature is usually permitted in certain specific programming situations, such as exception handling, etc.

Very soon several languages such as PASCAL, MODULA, C, etc., became available which were specifically designed to support structured programming. These programming languages facilitated writing modular programs and programs having good control structures. Therefore, messy control structure was no longer a big problem. So, the focus shifted from designing good control structures to designing good data structures for programs.

1.4.4 Data Structure-oriented Design

Computers became even more powerful with the advent of *integrated circuits* (ICs) in the early seventies. These could now be used to solve more complex problems. Software developers were tasked to develop larger and more complicated software. which often required writing in excess of several tens of thousands of lines of source code. The control flow-based program development techniques could not be used satisfactorily any more to write those programs, and more effective program development techniques were needed.

It was soon discovered that while developing a program, it is much more important to pay attention to the design of the important data structures of the program than to the design of its control structure. Design techniques based on this principle are called *data structure-oriented* design techniques.



1.4.6 Object-oriented Design

Data flow-oriented techniques evolved into *object-oriented design* (OOD) techniques in the late seventies. Object-oriented design technique is an intuitively appealing approach, where

Software Life Cycle Models

SDLC is a systematic process for building software that ensures the quality and correctness of the software built. SDLC process aims to produce high-quality software that meets customer expectations. The system development should be complete in the pre-defined time frame and cost. SDLC consists of a detailed plan which explains how to plan, build, and maintain specific software. Every phase of the SDLC life Cycle has its own process and deliverables that feed into the next phase. SDLC stands for Software Development Life Cycle and is also referred to as the Application Development life-cycle.

- It offers a basis for project planning, scheduling, and estimating
- Provides a framework for a standard set of activities and deliverables
- It is a mechanism for project tracking and control
- Increases visibility of project planning to all involved stakeholders of the development process
- Increased and enhance development speed
- Improved client relations
- Helps you to decrease project risk and project management plan overhead

SDLC Phases

The entire SDLC process divided into the following SDLC steps:



SDLC Phases

- Phase 1: Requirement collection and analysis
- Phase 2: Feasibility study
- Phase 3: Design
- Phase 4: Coding
- Phase 5: Testing
- Phase 6: Installation/Deployment
- Phase 7: Maintenance

In this tutorial, I have explained all these Software Development Life Cycle Phases

Phase 1: Requirement collection and analysis

The requirement is the first stage in the SDLC process. It is conducted by the senior team members with inputs from all the stakeholders and domain experts in the industry. Planning for the quality assurance requirements and recognization of the risks involved is also done at this stage.

This stage gives a clearer picture of the scope of the entire project and the anticipated issues, opportunities, and directives which triggered the project.

Requirements Gathering stage need teams to get detailed and precise requirements. This helps companies to finalize the necessary timeline to finish the work of that system.

Phase 2: Feasibility study

Once the requirement analysis phase is completed the next sdlc step is to define and document software needs. This process conducted with the help of 'Software Requirement Specification' document also known as 'SRS' document. It includes everything which should be designed and developed during the project life cycle.

There are mainly five types of feasibilities checks:

- **Economic:** Can we complete the project within the budget or not?
- **Legal:** Can we handle this project as cyber law and other regulatory framework/compliances.
- **Operation feasibility:** Can we create operations which is expected by the client?
- **Technical:** Need to check whether the current computer system can support the software
- **Schedule:** Decide that the project can be completed within the given schedule or not.

Phase 3: Design

In this third phase, the system and software design documents are prepared as per the requirement specification document. This helps define overall system architecture.

This design phase serves as input for the next phase of the model.

There are two kinds of design documents developed in this phase:

High-Level Design (HLD)

- Brief description and name of each module

- An outline about the functionality of every module
- Interface relationship and dependencies between modules
- Database tables identified along with their key elements
- Complete architecture diagrams along with technology details

Low-Level Design (LLD)

- Functional logic of the modules
- Database tables, which include type and size
- Complete detail of the interface
- Addresses all types of dependency issues
- Listing of error messages
- Complete input and outputs for every module

Phase 4: Coding

Once the system design phase is over, the next phase is coding. In this phase, developers start build the entire system by writing code using the chosen programming language. In the coding phase, tasks are divided into units or modules and assigned to the various developers. It is the longest phase of the Software Development Life Cycle process.

In this phase, Developer needs to follow certain predefined coding guidelines. They also need to use programming tools like compiler, interpreters, debugger to generate and implement the code.

Phase 5: Testing

Once the software is complete, and it is deployed in the testing environment. The testing team starts testing the functionality of the entire system. This is done to verify that the entire application works according to the customer requirement.

During this phase, QA and testing team may find some bugs/defects which they communicate to developers. The development team fixes the bug and send back to QA for a re-test. This process continues until the software is bug-free, stable, and working according to the business needs of that system.

Phase 6: Installation/Deployment

Once the software testing phase is over and no bugs or errors left in the system then the final deployment process starts. Based on the feedback given by the project manager, the final software is released and checked for deployment issues if any.

Phase 7: Maintenance

Once the system is deployed, and customers start using the developed system, following 3 activities occur

- Bug fixing – bugs are reported because of some scenarios which are not tested at all
- Upgrade – Upgrading the application to the newer versions of the Software
- Enhancement – Adding some new features into the existing software

The main focus of this SDLC phase is to ensure that needs continue to be met and that the system continues to perform as per the specification mentioned in the first phase.

Software modeling is the process of creating abstract representations of a software system. These models serve as blueprints that guide developers, designers, and stakeholders through the system's structure, behavior, and functionality.

By using diagrams and various modeling languages, software modeling helps in visualizing and understanding the complex aspects of the software, making it easier to plan, develop, and manage the system.

Choosing the right model is very important for the development of a software product or application. Development and testing processes are carried out based on the model.

There are four kinds of models

1. **Waterfall Model**
2. **RAD Model(Rapid Application Development)**
3. **Spiral Model**
4. **Agile Model**

1. Waterfall Model

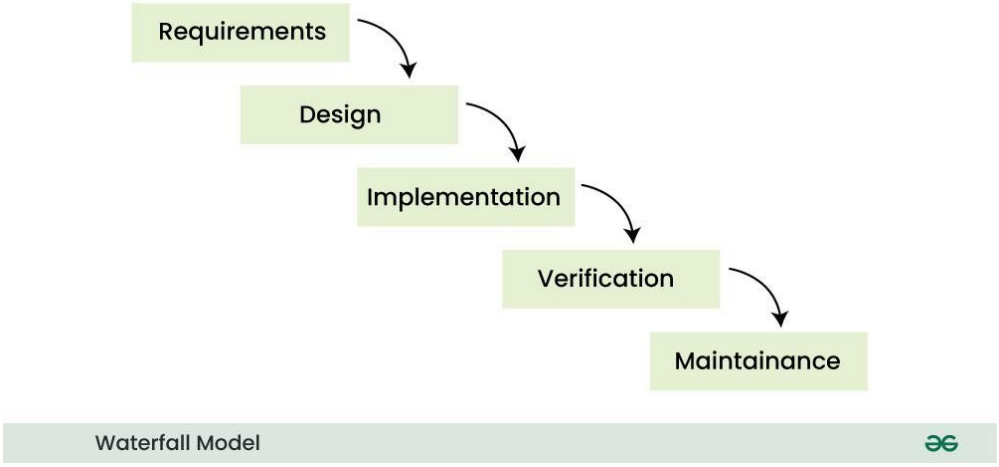
Waterfall model is a famous and good version of SDLC(System Development Life Cycle) for software engineering. The waterfall model is a linear and sequential model, which means that a development phase cannot begin until the previous phase is completed. We cannot overlap phases in waterfall model.

We can imagine waterfall in the following way

"Once the water starts flowing over the edge of the cliff, it starts falling down the mountain and the water cannot go back up."

Similarly waterfall model also works, once one phase of development is completed then we move to the next phase but cannot go back to the previous phase. In the waterfall model, the output of one phase serves as the input for the other phase.

Phases of Waterfall model



Waterfall Model

1. **Requirement phase:-** Requirement phase is the first phase of the waterfall model. In this phase the requirements of the system are collected and documented. This phase is very crucial because the next phases are based on this phase.
2. **Design phase:-** Design phase is based on the fact how the software will be built. The main objective of the design phase is to prepare the blueprint of the software system so that no problems are faced in the coming phases and solutions to all the requirements in the requirement phase are found.
3. **Implementation phase:-** In this phase, hardware, software and application programs are installed and the database design is implemented. Before the database design can be implemented, the software has to go through a testing, coding, and debugging process. This is the longest lasting phase in waterfall.
4. **Verification phase:-** In this phase the software is verified and it is evaluated that we have created the right product. In this phase, various types of testing are done and every area of the software is checked. It is believed that if we do not verify the software properly and there is any defect in it then no one will use it, hence verification is very important. One advantage of verification is that it reduces the risk of software failure.
5. **Maintenance phase:-** This is the last phase of waterfall. When the system is ready and users start using it, then the problems that arise have to be solved time-to-time. Taking care of the finished software and maintaining it as per time is called maintenance.

Advantages of Waterfall Model

- This model is simple and easy to understand.
- This is very useful for small projects.
- This model is easy to manage.
- The end goal is determined early.
- Each phase of this model is well explained.
- It provides a structured way to do things.

- This is a base model, all the SDLC models that came after this were created keeping this in mind, although they worked to remove its shortcomings.
- In this model, we can move to the next phase only after the first phase is successfully completed so that there is no overlapping between the phases.

Disadvantages of Waterfall Model

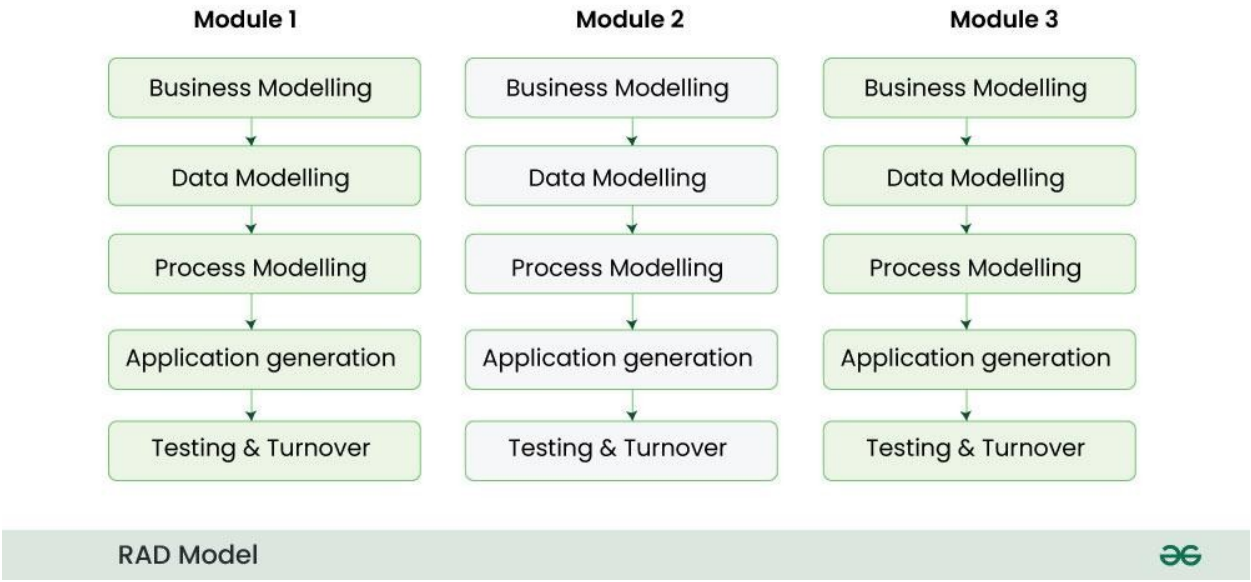
- In this model, complete and accurate requirements are expected at the beginning of the development process.
- Working software is not available for very long during the development life cycle.
- We cannot go back to the previous phase due to which it is very difficult to change the requirements.
- Risk is not assessed in this, hence there is high risk and uncertainty in this model.
- In this the testing period comes very late.
- Due to its sequential nature this model is not realistic in today's world.
- This is not a good model for large and complex projects.

RAD Model

RAD model stands for rapid application development model. The methodology of RAD model is similar to that of incremental or waterfall model. It is used for small projects.

- If the project is large then it is divided into many small projects and these small projects are planned one by one and completed. In this way, by completing small projects, the large project gets ready quickly.
- In RAD model, the project is completed within the given time and all the requirements are collected before starting the project. It is very fast and there are very less errors in it.

The main objective of RAD model is to reuse code, components, tools, processes in project development.



RAD Model

Phases RAD model

1. **Business modeling:** In this phase, the business model is designed on the basis of whatever functions the business has. If we speak in a little technical language, then we design the business model for the product on the basis of flow of information and distribution of information between different business channels. Here information flow means what type of information drives the business, where the information comes from and where it goes, who generates it, etc. This means that a complete business analysis is done in this phase.
2. **Data modeling:** Using the business model we had prepared, the data objects required for the business are defined.
3. **Process modeling:** The data objects that we defined in the data modeling phase are converted to establish the business information flow. It is necessary to achieve specific business objectives.
4. **Application generation:** In this phase we start building the software based on the output of the above three phases. For this we take the help of

automation tools. However, in this phase we do not develop the actual software but make a working prototype.

5. **Testing and turnover:** Whatever prototype we have prepared or whatever components and interfaces we have, they are tested in this phase. Since prototypes are tested separately during each iteration, the overall testing time in rapid application development is reduced.

Advantage of RAD Model

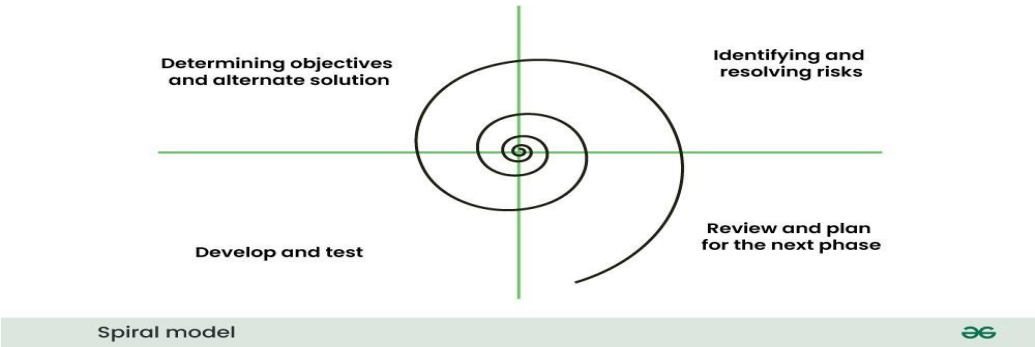
- It reduces the time taken in development.
- In this the components are reused.
- It is flexible and it is easy to make any changes in it.
- It is easy to transfer like scripts because high level abstraction and intermediate codes are used in it.
- There are very few defects in it because it is a prototype by nature.
- In this, productivity can be increased in less time with less people.
- It is cost effective.
- It is suitable for small projects.

Disadvantages of RAD Model

- In this we need highly skilled developers and designers.
- It is very difficult to manage.
- It is not suitable for project that are complex and takes long time.
- In this, feedback from the client is required for the development of each phase.
- Automated code generation is very expensive.
- This model is suitable only for component based and scalable systems.

Spiral Model

Spiral model is a software development process model. This model has characteristics of both iterative and waterfall models. This model is used in projects which are large and complex. This model was named spiral because if we look at its figure, it looks like a spiral, in which a long curved line starts from the center point and makes many loops around it. The number of loops in the spiral is not decided in advance but it depends on the size of the project and the changing requirements of the user. We also call each loop of the spiral a phase of the software development process.
A software project goes through these loops again and again in iterations. After each iteration a more and more complete version of the software is developed. The most special thing about this model is that risks are identified in each phase and they are resolved through prototyping. This feature is also called Risk Handling.



Spiral Model

In Spiral Model the entire process of software development is described in four phases which are repeated until the project is completed.
Those phases are as follows:-

- **Determining objectives and alternate solutions**: In the first phase, whatever requirements the customer has related to the software are collected. On the basis of which objectives are identified and analyzed and various alternative solutions are proposed.

- **Identifying and resolving risks**: In this phase, all the proposed solutions are assessed and the best solution is selected. Now that solution is analyzed and the risks related to it are identified. Now the identified risks are resolved through some best strategy.
- **Develop and test**: Now the development of software is started. In this phase various features are implemented, that is, their coding is done. Then those features are verified through testing.
- **Review and plan for the next phase:** In this phase the developed version of the software is given to the customer and he evaluates it. Gives his feedback and tells new requirements. Finally planning for the next phase (next spiral) is started.

Advantages of Spiral Model

- If we have to add additional functionality or make any changes to the software, then through this model we can do so in the later stages also.
- Spiral model is suitable for large and complex projects.
- It is easy to estimate how much the project will cost.
- Risk analysis is done in each phase of this model.
- The customer can see the look of his software only in the early stages of the development process.
- Since continuous feedback is taken from the customer during the development process, the chances of customer satisfaction increases.

Disadvantage of Spiral Model

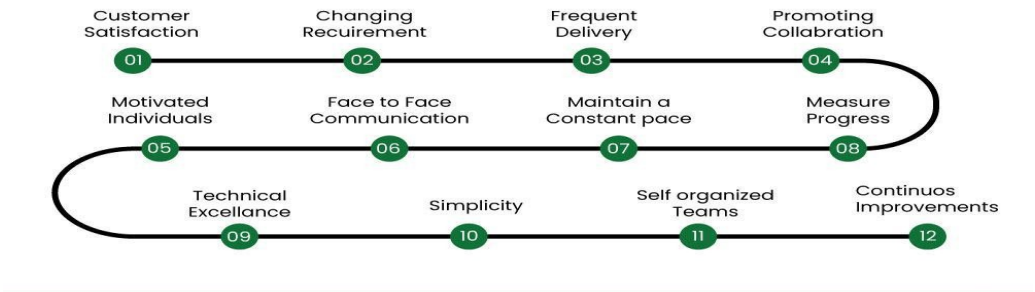
- This is the most complex model of SDLC, due to which it is quite difficult to manage.
- This model is not suitable for small projects.
- The cost of this model is quite high.
- It requires more documentation than other models.
- Experienced experts are required to evaluate and review the project from time to time.
- Using this model, the success of the project depends greatly on the risk analysis phase.

Agile Model

Agile Software Development is a software development methodology that values flexibility, collaboration, and customer satisfaction. It is based on the Agile Manifesto, a set of principles for software development that prioritize individuals and interactions, working software, customer collaboration, and responding to change.
Agile Software Development is an iterative and incremental approach to software development that emphasizes the importance of delivering a working product quickly and frequently. It involves close collaboration between the development team and the customer to ensure that the product meets their needs and expectations.

Principles of Agile Software Development

The Agile Manifesto is based on four values and twelve principles that form the basis, for methodologies.



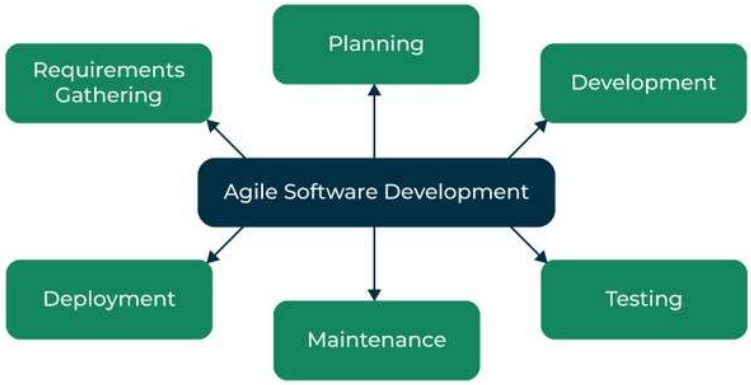
12 Principles of Agile Methodology

These principles include:

1. Ensuring customer satisfaction through the early delivery of software.
2. Being open to changing requirements in the stages of the development.
3. Frequently delivering working software with a main focus on preference for timeframes.
4. Promoting collaboration between business stakeholders and developers as an element.
5. Structuring the projects around individuals. Providing them with the necessary environment and support.
6. Prioritizing face to face communication whenever needed.
7. Considering working software as the measure of the progress.
8. Fostering development by allowing teams to maintain a pace indefinitely.
9. Placing attention on excellence and good design practices.

10. Recognizing the simplicity as crucial factor aiming to maximize productivity by minimizing the work.
11. Encouraging self organizing teams as the approach to design and build systems.
12. Regularly reflecting on how to enhance effectiveness and to make adjustments accordingly.

The Agile Software Development Process



Agile Software Development

1. **Requirements Gathering:** The customer's requirements for the software are gathered and prioritized.
2. **Planning:** The development team creates a plan for delivering the software, including the features that will be delivered in each iteration.
3. **Development:** The development team works to build the software, using frequent and rapid iterations.
4. **Testing:** The software is thoroughly tested to ensure that it meets the customer's requirements and is of high quality.
5. **Deployment:** The software is deployed and put into use.
6. **Maintenance:** The software is maintained to ensure that it continues to meet the customer's needs and expectations.

Agile Software Development is widely used by software development teams and is considered to be a flexible and adaptable approach to software development that is well-suited to changing requirements and the fast pace of software development.

Agile is a time-bound, iterative approach to software delivery that builds software incrementally from the start of the project, instead of trying to deliver all at once.

Design Process of Agile software Development

- In Agile development, Design and Implementation are considered to be the central activities in the software process.
- The design and Implementation phase also incorporates other activities such as requirements elicitation and testing.
- In an agile approach, iteration occurs across activities. Therefore, the requirements and the design are developed together, rather than separately.
- The allocation of requirements and the design planning and development as executed in a series of increments. In contrast with the conventional model, where requirements gathering needs to be completed to proceed to the design and development phase, it gives Agile development an extra level of flexibility.
- An agile process focuses more on code development rather than documentation.

UNIT-II

Software Project Management: Software project management complexities, Responsibilities of a software project manager, Metrics for project size estimation, Project estimation techniques, Empirical Estimation techniques, COCOMO, risk management.

Requirements Analysis and Specification: Requirements gathering and analysis, Software Requirements Specification (SRS), Formal system specification, Axiomatic specification, Algebraic specification, Executable specification and 4GL.

Software Project Management Complexities refer to the various difficulties to manage a software project. It recognizes in many different ways. The main goal of software project management is to enable a group of developers to work effectively toward the successful completion of a project in a given time. But software project management is a very difficult task.

Earlier many projects have failed due to faulty project management practices. Management of software projects is much more complex than management of many other types of projects. In this article, we will discuss the types of Complexity as well as the factors that make Project Management Complex.

Types of Complexity

The following are the types of complexity in software project management:

- **Time Management Complexity**: Complexities to estimate the duration of the project. It also includes the complexities to make the schedule for different activities and timely completion of the project.

- **Cost Management Complexity**: Estimating the total cost of the project is a very difficult task and another thing is to keep an eye that the project does not overrun the budget.
- **Quality Management Complexity:** The quality of the project must satisfy the customer's requirements. It must assure that the requirements of the customer are fulfilled.
- **Risk Management Complexity**: Risks are the unanticipated things that may occur during any phase of the project. Various difficulties may occur to identify these risks and make amendment plans to reduce the effects of these risks.
- **Human Resources Management Complexity**: It includes all the difficulties regarding organizing, managing, and leading the project team.
- **Communication Management Complexity**: All the members must interact with all the other members and there must be good communication with the customer.
- **Deployment complexity**: A release candidate, or finalized code, has to be synchronized from one system to another. Conceptually, such an operation ought to be simple. To perform this synchronization swiftly and securely in practice proves to be difficult.
- **Procurement Management Complexity**:
- Projects need many services from third parties to complete the task. These may increase the complexity of the project to acquire the services.
- **Integration Management Complexity**: The difficulties regarding coordinating processes and developing a proper project plan. Many changes may occur during the project development and it may hamper the project completion, which increases the complexity.

Responsiblities of a Project Manager

A project manager has to face many difficult situations to accomplish these works. The job responsibilities of a project manager range from invisible activities like building up team morale to highly visible customer presentations. Most of the managers take responsibility for writing the project proposal, project cost estimation, scheduling, project staffing, software process tailoring, project monitoring and control, software configuration management, risk management, managerial report writing, and presentation, and interfacing with clients.

The tasks of a project manager are classified into two major types:

1. Project planning
2. Project monitoring and control

Project Planning

Project planning is undertaken immediately after the feasibility study phase and before the starting of the requirement analysis and specification phase. Once a project is feasible, Software project managers start project planning. Project planning is completed before any development phase starts.

1. Project planning involves estimating several characteristics of a project and then plan the project activities based on these estimations.
2. Project planning is done with most care and attention.
3. A wrong estimation can result in schedule slippage.
4. Schedule delay can cause customer dissatisfaction, which may lead to a project failure.
5. Before starting a software project, it is essential to determine the tasks to be performed and properly manage allocation of tasks among individuals involved is the software development.
6. Hence, planning is important as it results in effective software development.

7. Project planning is an organized and integrated management process, which focuses on activities required for successful completion of the project.
8. It prevents obstacles that arise in the project such as changes in projects or organizations objectives, non-availability of resources, and so on.
9. Project planning also helps in better utilization of resources and optimal usage of the allotted time for a project.

10.For effective project planning, in addition to a very good knowledge of various estimation techniques, experience is also very important.

Project Monitoring & Control

1. Project Estimation

Project Size Estimation is the most significant parameter based on which all other estimations like cost, duration and effort are made.

- **Cost Estimation:** Total expenses to develop the software product is estimated.
- **Time Estimation:** The total time required to complete the project.
- **Effort Estimation:** The effort needed to complete the project is estimated.

2. Scheduling

After the completion of the estimation of all the project parameters, scheduling for manpower and other resources is done.

3. Staffing

Team structure and staffing plans are made.

4. Risk Management

The project manager should identify the unanticipated risks that may occur during project development risk, analyze the damage that might cause these risks, and take a risk reduction plan to cope with these risks.

5. Miscellaneous Plans

This includes making several other plans such as quality assurance plans, configuration management plans, etc.

- **Lead the team:** The project manager must be a good leader who makes a team of different members of various skills and can complete their individual tasks.
- **Motivate the team-member:** One of the key roles of a software project manager is to encourage team members to work properly for the successful completion of the project.
- **Tracking the progress:** The project manager should keep an eye on the progress of the project. A project manager must track whether the project is going as per plan or not. If any problem arises, then take the necessary action to solve the problem. Moreover, check whether the product is developed by maintaining correct coding standards or not.
- **Monitoring and reviewing:** Project monitoring is a continuous process that lasts the whole time a product is being developed, during which the project manager compares actual progress and cost reports with anticipated reports as soon as possible. While most firms have a formal system in place to track progress, qualified project managers may still gain a good understanding of the project's development by simply talking with participants.
- **Documenting project report:** The project manager prepares the documentation of the project for future purposes. The reports contain detailed features of the product and various techniques. These reports help to maintain and enhance the quality of the project in the future.

- **Reporting:** Reporting project status to the customer and his or her organization is the responsibility of the project manager. Additionally, they could be required to prepare brief, well-organized pieces that summarize key details from in-depth studies.

What is Project Size Estimation?

Project size estimation is determining the scope and resources required for the project.

1. It involves assessing the various aspects of the project to estimate the effort, time, cost, and resources needed to complete the project.
2. Accurate project size estimation is important for effective and efficient project planning, management, and execution.